

Automated Deployment of Web Application Using AWS Code Pipeline

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Submitted by

SHAIK SHEEMA TABASUM 23HP1A4226, ALIET VIJAYAWADA (Batch -1)

PULAKANDAM HEMALATHA 23HP1A04209, ALIET VIJAYAWADA (Batch -1)

GOLLA TANUJA 23HP1A4233, ALIET VIJAYAWADA (Batch -1)

RAMINENI REVANTHKUMAR 23HP1A05J0, ALIET VIJAYAWADA (Batch -1)

SEERAPU MURALI KRISHNA 24HP1A0551, ALIET VIJAYAWADA (Batch -1)

YANAMALAMANDA MANOJ KUMAR 24HP1A4249, ALIET VIJAYAWADA (Batch-1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Anil Kumar Varanasi, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Shaik Sheema Tabasum, sheematabasum2006@gmail.com.
Busetty Guru Chaitanya Vardhan, S200720, chaithu2712@gmail.com.
Pasupula Jyothi Kumar Reddy, S200748, jyothikumarreddy2005@gmail.com.
N Manjunath, S200727 , nmanjul612@gmail.com.
M Yoga Susmitha, S200718 , mallisettysusmitha99@gmail.com.

ABSTRACT

In modern software development, rapid and reliable application deployment is essential for delivering high-quality services. This project demonstrates the implementation of an automated deployment pipeline for a web application using Amazon Web Services (AWS). The primary objective is to eliminate manual deployment processes by integrating Continuous Integration and Continuous Deployment (CI/CD) practices. The project involves developing a static DevOps Project Showcase Website and hosting it on Amazon S3. The source code is maintained in a GitHub repository, which serves as the version control system. AWS CodePipeline is configured to automatically detect code changes in the repository and initiate the deployment workflow. AWS CodeBuild is used to build and prepare the application before deployment. Once the process is completed, the updated website is automatically deployed to Amazon S3 and made available to users.

The project involves developing a static DevOps Project Showcase Website and hosting it on Amazon S3. The source code is maintained in a GitHub repository, which serves as the version control system. AWS CodePipeline is configured to automatically detect code changes in the repository and initiate the deployment workflow. AWS CodeBuild is used to build and prepare the application before deployment. Once the process is completed, the updated website is automatically deployed to Amazon S3 and made available to users.

The successful implementation of this project showcases how AWS services can be leveraged to create a scalable, reliable, and automated deployment solution for modern web applications while reducing operational overhead and enhancing productivity.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION	5
2.	METHODOLOGY	6
3.	TECHNOLOGY STACK	9
4.	SYSTEM DESIGN / ARCHITECTURE.....	11
5.	IMPLEMENTATION	23
6.	RESULTS	26
7.	CONCLUSION.....	29

1. INTRODUCTION

In today's rapidly evolving software industry, organizations require faster, more reliable, and efficient methods for delivering applications to users. Traditional deployment processes often involve manual intervention, which can lead to delays, inconsistencies, and human errors. To overcome these challenges, modern development teams adopt DevOps practices that integrate software development and operations to achieve continuous delivery and improved collaboration.

The web application source code is maintained in a GitHub repository, which acts as the version control system. Whenever changes are pushed to the repository, AWS CodePipeline automatically detects the updates and initiates the deployment workflow. AWS CodeBuild is used to build and prepare the application, while Amazon S3 serves as the hosting platform for the static website. This automated process ensures that the latest version of the application is deployed quickly and consistently.

The project also highlights the practical implementation of Cloud Computing, Version Control, Automation, and CI/CD Pipelines in a real-world environment. By utilizing AWS services, the deployment process becomes scalable, reliable, and cost-effective.

This project provides valuable insights into modern software deployment practices and demonstrates how DevOps methodologies can improve productivity, reduce deployment time, and enhance overall software quality.

This project, "Automated Deployment of Web Application Using AWS CodePipeline," demonstrates the implementation of a Continuous Integration and Continuous Deployment (CI/CD) pipeline for a web application. The project focuses on automating the deployment process using Amazon Web Services (AWS), reducing the need for manual deployment and ensuring faster application updates.

2. METHODOLOGY

The methodology followed in this project focuses on implementing an automated deployment process for a web application using AWS services and DevOps practices. The project is carried out through the following phases:

1. Requirement Analysis

The project requirements were identified, including the development of a static web application and the implementation of an automated deployment pipeline. The objective was to eliminate manual deployment and ensure continuous delivery of updates.

2. Website Development

A DevOps Project Showcase Website was developed using HTML, CSS, and JavaScript. The website contains information about DevOps concepts, AWS services, project architecture, team details, and deployment workflow.

3. Version Control Using GitHub

The website source code was stored in a GitHub Repository. GitHub acts as the centralized version control system where all project files and updates are maintained. Team members can collaborate efficiently by pushing code changes to the repository.

4. AWS S3 Configuration

An Amazon S3 Bucket was created and configured for Static Website Hosting. The website files are deployed to the S3 bucket, making the website accessible through a public URL.

5. AWS CodeBuild Setup

AWS CodeBuild was configured to build and validate the application whenever new code is pushed to the GitHub repository. This ensures that the application is prepared for deployment automatically.

6. AWS CodePipeline Integration

AWS CodePipeline was configured to automate the deployment process. The pipeline continuously monitors the GitHub repository for changes. Whenever code updates are detected, CodePipeline automatically triggers the build and deployment stages.

7. Automated Deployment Process

The deployment workflow follows these steps:

Developer → GitHub → AWS CodePipeline → AWS CodeBuild → Amazon S3 → Live Website

Whenever code is committed and pushed to GitHub, the pipeline automatically executes the build and deployment process without manual intervention.

8. Testing and Validation

The deployment pipeline was tested by making changes to the website source code and pushing them to GitHub. Successful deployment was verified by checking whether the updated content appeared on the live website hosted on Amazon S3.

9. Monitoring and Maintenance

The deployment process was monitored using AWS services to ensure successful execution of each stage. Any errors or failures were identified and resolved to maintain continuous availability of the website.

3. TECHNOLOGY STACK

The Serverless Inventory Management System was developed using a modern, cloud-native technology stack that supports event-driven architecture, real-time processing, and scalability.

The following technologies and services were used throughout the implementation:

3.1. Cloud Platform

- **Amazon Web Services (AWS):** The entire project is hosted on AWS, leveraging its serverless services for storage, computation, database management, and notifications.

3.2. AWS Services

Amazon S3: Used for storing input CSV files. Triggers Lambda functions on file uploads.

- **AWS Lambda:** Executes the core business logic — parsing files, processing data, updating the database, and triggering notifications.
- **Amazon DynamoDB:** NoSQL database for storing product names, quantities, and timestamps. Offers fast and scalable access to inventory records.

- **Amazon SNS (Simple Notification Service):** Sends real-time email alerts when low-stock products are detected.
- **AWS IAM (Identity and Access Management):** Provides secure access control between AWS services by defining roles and permissions.
- **Amazon CloudWatch:** Monitors the system's performance and logs Lambda executions for debugging and optimization.

3.3. Programming & Scripting

- **Python 3.12 (Lambda Runtime):** Used for writing the serverless backend logic in AWS Lambda functions.
- **Boto3 SDK:** Python SDK for AWS used within Lambda to interact with S3, DynamoDB, and SNS.

3.4. Data Format

CSV (Comma-Separated Values): Used as the input format for inventory data uploads.

3.5. Security & Access

IAM Roles and Policies: Configured to allow only the necessary permissions for each service, following the principle of least privilege.

3.6. Monitoring & Logging

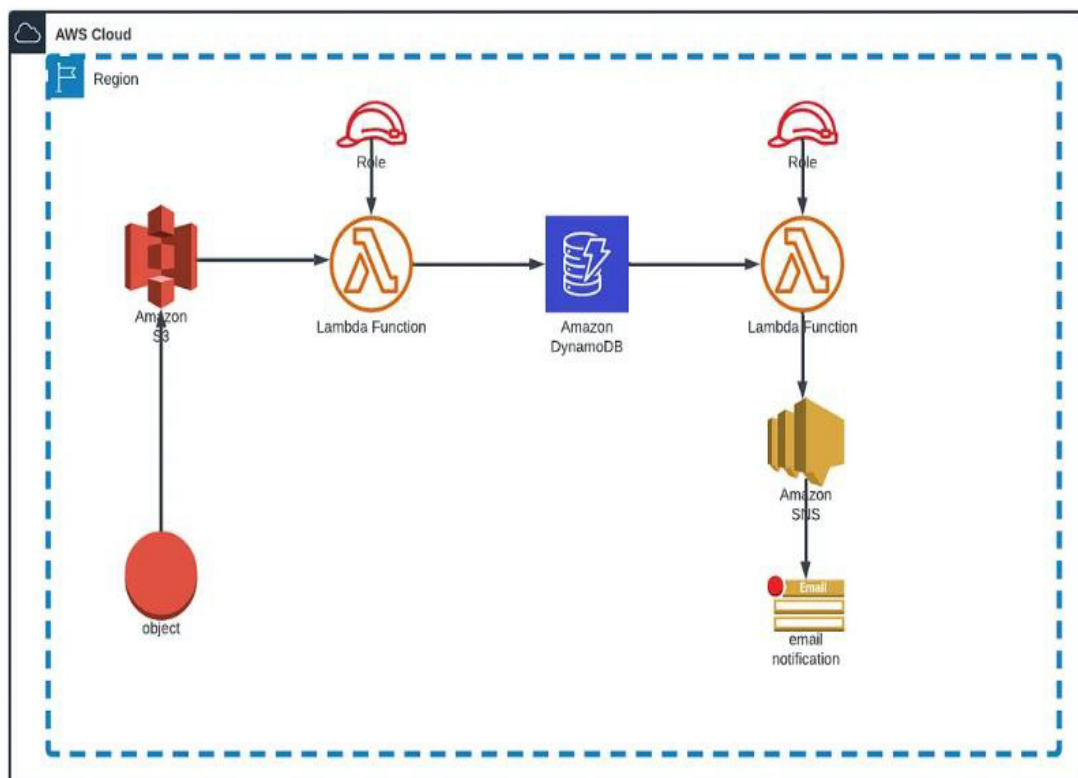
- **CloudWatch Logs:** Captures execution logs from Lambda for tracking and troubleshooting.

- **CloudWatch Metrics:** Provides real-time data on function invocations, errors, and duration.

4. SYSTEM DESIGN / ARCHITECTURE

The Serverless Inventory Management System is designed as an event-driven, cloud-native solution that automates inventory processing and low-stock notifications. The architecture adopts a **serverless model**, ensuring scalability, minimal infrastructure maintenance, and cost-efficiency. The system follows a multi-tier design, comprising the **data ingestion layer**, **processing logic**, **data storage**, and **notification layer**, all powered by managed AWS services.

This section outlines the core components and their interactions in the architecture.



4.1. Data Ingestion Layer – Amazon S3

The process begins when a user uploads a **CSV file** containing product details (e.g., Product Name, Quantity) to an **Amazon S3 bucket**. S3 acts as the storage layer for input files. An **S3 event trigger** is configured to detect new object uploads and invoke the processing component automatically.

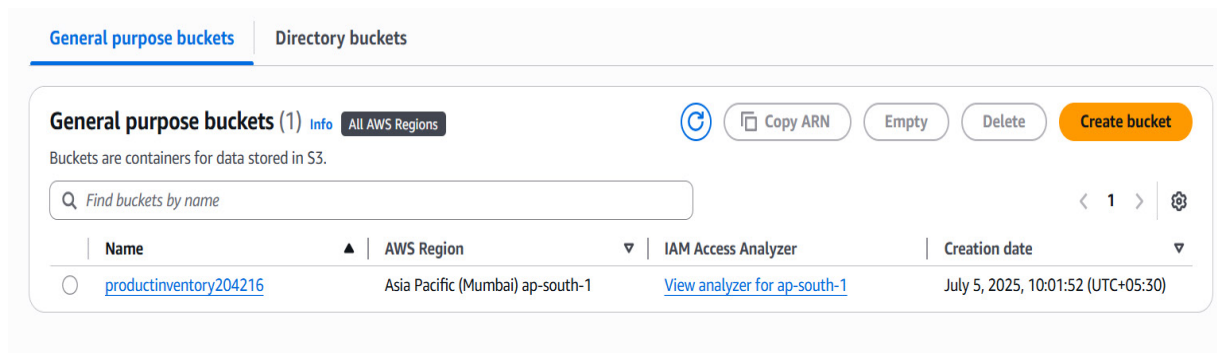


Figure 1.bucket name

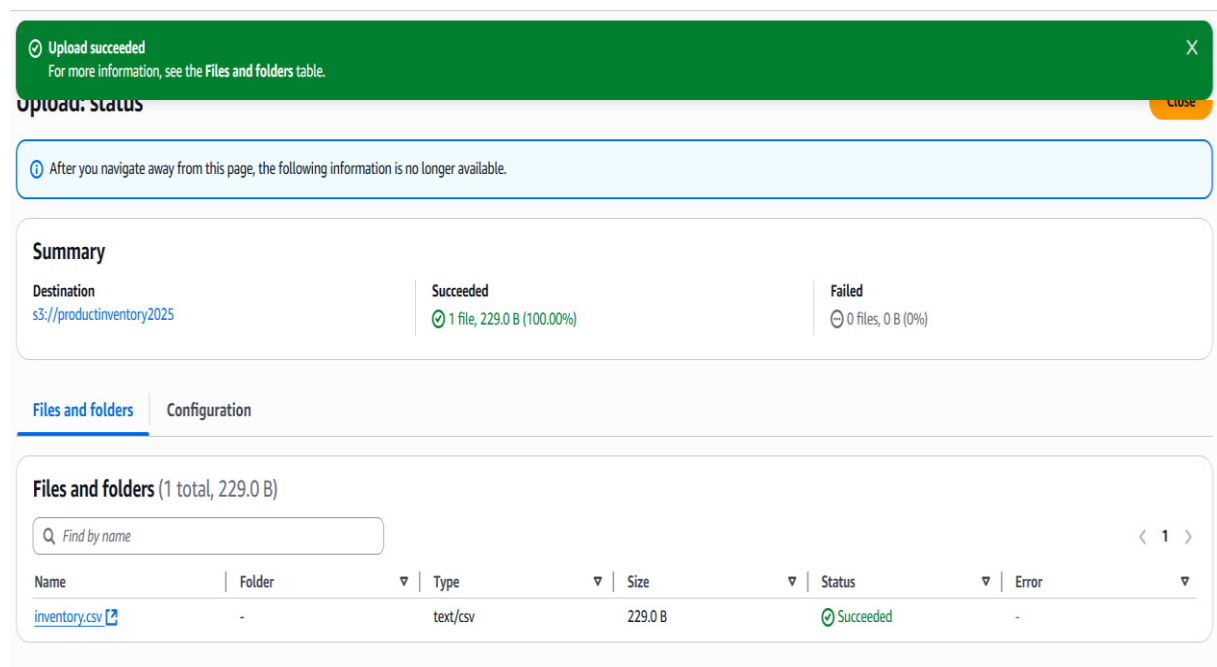


Figure 2.csv file uploaded into s3 bucket

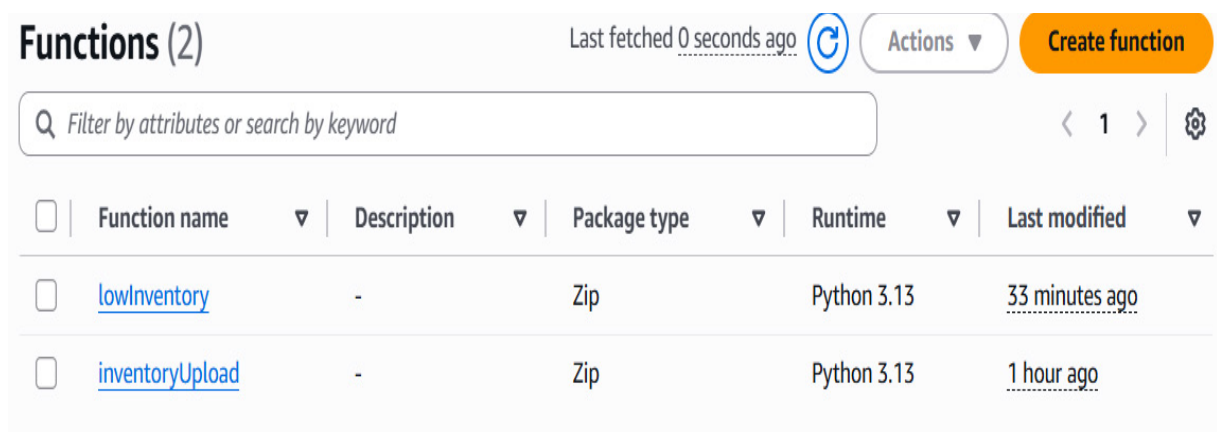
Key Features :

- Supports various file formats (e.g., .csv)
- Triggers Lambda functions upon file upload
- Scalable and cost-effective storage

4.2. Processing Logic – AWS Lambda

An **AWS Lambda function** is triggered by the S3 event. This function performs multiple tasks:

- Reads and parses the CSV file
- Validates product and quantity data
- Uploads each product record into **Amazon DynamoDB**
- Scans the table to identify products below a minimum quantity threshold
- Sends an email summary of low-stock items via **Amazon SNS**



The screenshot shows the AWS Lambda console interface. At the top, it says 'Functions (2)' with a refresh button and 'Last fetched 0 seconds ago'. There is a search bar with the placeholder 'Filter by attributes or search by keyword'. To the right of the search bar are navigation controls showing '< 1 >' and a settings gear icon. Below this is a table with two functions listed. The table has columns for checkboxes, Function name, Description, Package type, Runtime, and Last modified. The first function is 'lowInventory' with a description of '-', package type 'Zip', runtime 'Python 3.13', and last modified '33 minutes ago'. The second function is 'inventoryUpload' with a description of '-', package type 'Zip', runtime 'Python 3.13', and last modified '1 hour ago'.

☐	Function name	Description	Package type	Runtime	Last modified
☐	lowInventory	-	Zip	Python 3.13	33 minutes ago
☐	inventoryUpload	-	Zip	Python 3.13	1 hour ago

Figure 3.Functions In the Lambda

inventoryUpload

▼ Function overview [Info](#)

Diagram

Template



inventoryUpload



Layers

(0)



S3

+ Add trigger

Figure 4.Design of inventoryUpload Function

Code

Test

Monitor

Configuration

Aliases

Versions

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Triggers (1) [Info](#)

[Fix errors](#) [Edit](#) [Delete](#) [Add trigger](#)

☐

Trigger

☐

S3: productinventory204216
arn:aws:s3:::productinventory204216
[► Details](#)

Figure 5.Triggers in inventoryUpload Function

Trigger configuration

 S3
aws asynchronous storage

Bucket
Choose or enter the ARN of an S3 bucket that serves as the event source. The bucket must be in the same region as the function.



Bucket must be in region ap-south-1

Event types
Select the events that you want to have trigger the Lambda function. You can optionally set up a prefix or suffix for an event. However, for each bucket, individual events cannot have multiple configurations with overlapping prefixes or suffixes that could match the same object key.



Prefix - optional
Enter a single optional prefix to limit the notifications to objects with keys that start with matching characters. Any [special characters](#) must be URL encoded.

Suffix - optional
Enter a single optional suffix to limit the notifications to objects with keys that end with matching characters. Any [special characters](#) must be URL encoded.

Lambda will add the necessary permissions for AWS S3 to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

Figure 6.S3 bucket Trigger Configuration in inventoryUpload Function

 [Lambda](#) > [Functions](#) > [inventoryUpload](#) > [Edit basic settings](#)

Your function is allocated CPU proportional to the memory configured.

MB
Set memory to between 128 MB and 10240 MB

Ephemeral storage | [Info](#)
You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#)

MB
Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart | [Info](#)
Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#). For Python and .NET runtimes, [view pricing](#).

Supported runtimes: .NET 8 (C#/F#/PowerShell), Java 11, Java 17, Java 21, Python 3.12, Python 3.13.

Timeout
 min sec

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.



[View the InventoryLambdaExecutionRole role](#) on the IAM console.

Figure 7.inventoryUpload Basic Settings

▼ Function overview [Info](#)

Diagram

Template

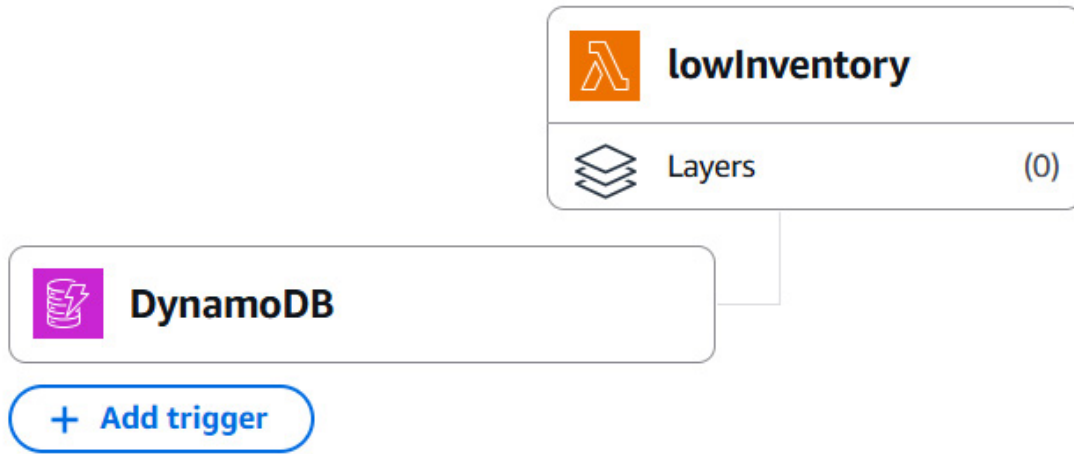


Figure 8.Design of lowInventory Function

[Lambda](#) > [Functions](#) > [lowInventory](#) > [Edit trigger](#)

Edit trigger

Trigger configuration

DynamoDB
aws database event-source-mapping nosql polling

DynamoDB table

Choose or enter the ARN of a DynamoDB table.

`arn:aws:dynamodb:ap-south-1:888947293269:table/AppleInventoryTable/stream/2025-07-05T03:46:52.882`

Event poller configuration

☐ **Activate trigger**

Select to activate the trigger now. Keep unchecked to create the trigger in a deactivated state for testing (recommended).

☐ **Enable metrics**

Monitor your event source with metrics. You can view those metrics in CloudWatch console. Enabling this feature incurs additional costs. [Learn more](#)

Batch size [Info](#)

The maximum number of records in each batch to send to the function.

1000

Figure 9.Dynamodb Trigger Configuration in lowInventory Function

Ephemeral storage [Info](#)
 You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#) [\[3\]](#)

MB
 Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart [Info](#)
 Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#) [\[3\]](#). For Python and .NET runtimes, [view pricing](#) [\[3\]](#).

Supported runtimes: .NET 8 (C#/F#/PowerShell), Java 11, Java 17, Java 21, Python 3.12, Python 3.13.

Timeout
 min sec

Execution role
 Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#) [\[3\]](#).

☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
 Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

[View the LowInventoryLambdaExecutionRole role](#) [\[3\]](#) on the IAM console.

Figure 10. lowInventory Function Basic Settings

4.3. Data Storage – Amazon DynamoDB

Product records are stored in **Amazon DynamoDB**, a fast, fully managed NoSQL database.

Each entry includes:

- ProductName (Primary Key)
- StoreRegion
- ProductCount

Tables (1) Info									
Find tables				Any tag key	Any tag value		1		
☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	
☐	AppleInventoryTable	Active	Product (S)	StoreRegion (S)	0	0	Off	☆	

Figure 11. Dynamodb Table

Completed · Items returned: **10** · Items scanned: **10** · Efficiency: **100%** · RCUs consumed: **2**

×

Table: AppleInventoryTable - Items returned (10)

Actions ▼

Create item

Scan started on July 05, 2025, 11:25:19

< 1 >

⚙️

☐	Product (String) ▼	StoreRegion (String) ▼	ProductCount ▼
☐	iPad	Kisumu	4
☐	Thunderbolt 3	Kiambu	7
☐	AirPods	Kiambu	2
☐	Apple Pencil	Kisumu	8
☐	MacBook Air	Nairobi	6
☐	Siri Remote	Nakuru	10
☐	iPhone	Nakuru	5
☐	Magic Mouse	Mombasa	9
☐	iPhone 12	Kisumu	3
☐	iPhone 12	Nairobi	1

Figure 12.Dynamodb Table items

4.4. Notification Layer – Amazon SNS

Once low-quantity items are identified, the Lambda function formats the result into a structured message. This message is published to an **Amazon SNS topic**, which is subscribed to by stakeholders via **email**. SNS ensures real-time delivery of alerts, allowing inventory managers to take immediate action.

Features:

- Supports multiple subscribers
- Fast and reliable push-based delivery
- Customizable message formatting



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

`arn:aws:sns:ap-south-1:888947293269:LowInventoryTopic:2632756f-5482-4a54-9648-67902fad5d2f`

If it was not your intention to subscribe, [click here to unsubscribe](#).

Figure 13.SNS Subscription

4.5. Security and Access Control – AWS IAM

AWS Identity and Access Management (IAM) is used to securely grant permissions between services:

- Lambda can read from S3 and write to DynamoDB
- Lambda can publish to SNS
- Least privilege access policies are enforced to enhance security

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management New

Roles (6) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

< 1 >

☐	Role name	Trusted entities	Last activity
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Linker)	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linker)	-
☐	InventoryLambdaExecutionRole	AWS Service: lambda	27 minutes ago
☐	inventoryUpload-role-ul9xj32	AWS Service: lambda	-
☐	lowInventory-role-sr25shmg	AWS Service: lambda	1 hour ago
☐	LowInventoryLambdaExecutionRole	AWS Service: lambda	30 minutes ago

Figure 14.Roles in IAM

Permissions policies (2) Info

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 >

☐	Policy name	Type	Attached entities
☐	AllowCloudWatchLogs	Customer inline	0
☐	LowInventoryLambdaExecutionRole	Customer managed	1

Figure 15.permission polices in LowInventoryLambdaExecutionRole

Permissions policies (1) Info

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 >

☐	Policy name	Type	Attached entities
☐	InventoryLambdaExecutionRole	Customer managed	1

Figure 16.permission polices in InventoryLambdaExecutionRole

Permissions defined in this policy [Info](#)

[Copy](#)[Edit](#)[Summary](#)[JSON](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "s3:GetObject",  
8         "s3:ListBucket"  
9       ],  
10      "Resource": [  
11        "arn:aws:s3:::productinventory204216/*",  
12        "arn:aws:s3:::productinventory204216"  
13      ]  
14    },  
15    {  
16      "Effect": "Allow",  
17      "Action": [  
18        "dynamodb:PutItem",  
19        "dynamodb:BatchWriteItem"  
20      ],  
21      "Resource": "arn:aws:dynamodb:ap-south-1:888947293269:table/AppleInventoryTable"  
22    }  
23  ]  
24 }
```

Figure 17.json code in InventoryLambdaExecutionRole policy

Permissions defined in this policy [Info](#)

[Copy](#)[Edit](#)[Summary](#)[JSON](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "dynamodb:PutItem",  
8         "dynamodb:UpdateItem",  
9         "dynamodb:GetItem",  
10        "dynamodb:Scan",  
11        "dynamodb:Query"  
12      ],  
13      "Resource": "arn:aws:dynamodb:ap-south-1:888947293269:table/AppleInventoryTable"  
14    },  
15    {  
16      "Effect": "Allow",  
17      "Action": [  
18        "dynamodb:GetRecords",  
19        "dynamodb:GetShardIterator",  
20        "dynamodb:DescribeStream",  
21        "dynamodb:ListStreams"  
22      ],  
23      "Resource": "arn:aws:dynamodb:ap-south-1:888947293269:table/AppleInventoryTable/stream/*"  
24    },  
25    {  
26      "Effect": "Allow",  
27      "Action": "sns:Publish",  
28      "Resource": "arn:aws:sns:ap-south-1:888947293269:LowInventoryTopic"  
29    }  
30  ]  
31 }
```

Figure 18.json code in LowInventoryLambdaExecutionRole policy

4.6. Monitoring and Logging – Amazon CloudWatch

Amazon CloudWatch is integrated for:

- Real-time monitoring of Lambda execution
- Viewing logs for debugging and error tracing
- Setting alerts for function failures or unusual activity

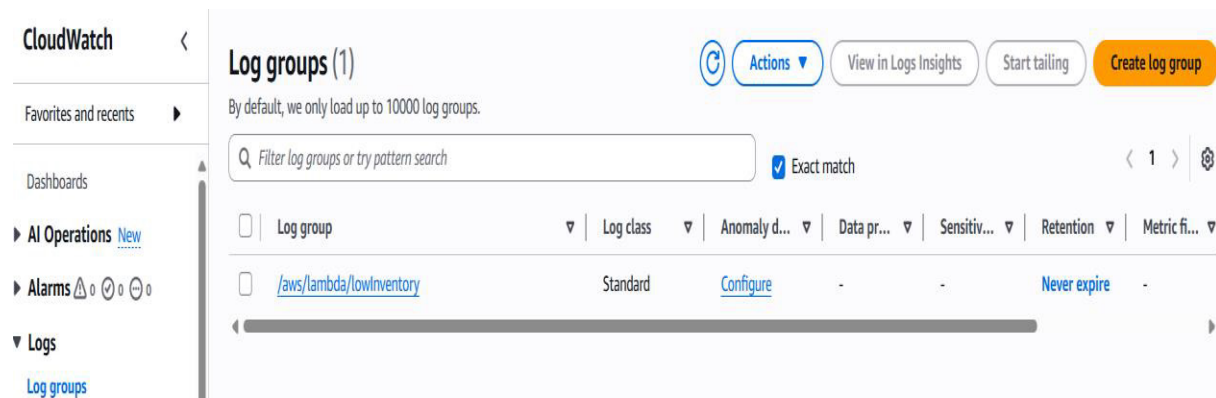


Figure 19. lowInventory function In CloudWatch

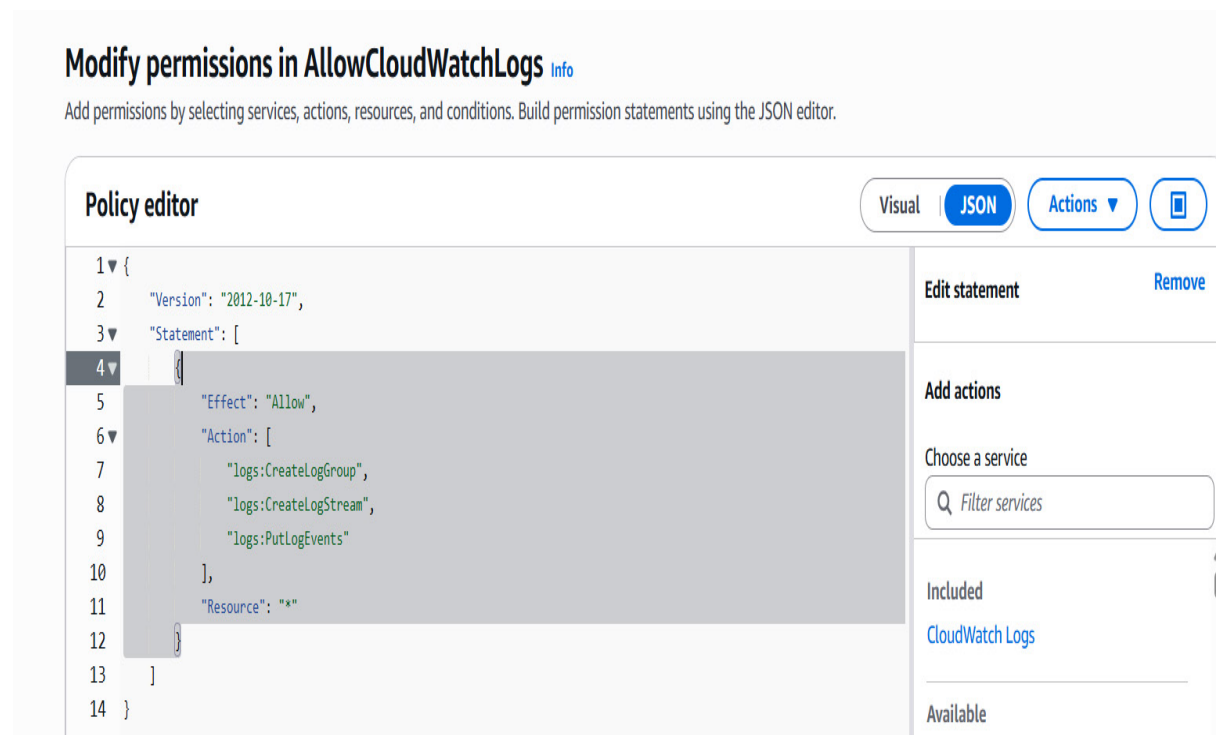


Figure 20. json inline code in CloudWatch Policy

5. IMPLEMENTATION

The implementation of the Serverless Inventory Management System translates the architectural design into a fully functional cloud-based solution. This section outlines how each AWS service and component was configured and developed to work together in an automated, event-driven workflow. The implementation focused on enabling real-time inventory updates, automated data processing, and alert mechanisms — all without managing any physical servers.

5.1. CSV File Upload to Amazon S3

The process begins with the user uploading a .csv file containing two columns: **Product Name** and **Quantity**. An **Amazon S3 bucket** is configured to serve as the input layer for file ingestion.

The S3 bucket has an **event notification** configured to trigger a Lambda function whenever a new file is uploaded.

5.2. AWS Lambda for File Processing and Business Logic

An **AWS Lambda function** (Python) is triggered upon the file upload. It performs several critical tasks:

1. **Reads the CSV file** from S3.
2. **Parses and validates** each row of the file.
3. **Uploads the data** (ProductName, Quantity, Timestamp) into the **DynamoDB** table.
4. **Scans** the table to check for products where the quantity is less than or equal to a threshold (e.g., 5).
5. **Sends an alert** via Amazon SNS with a list of low-quantity items.

Key Lambda details:

- Runtime: Python 3.12
- IAM role with S3 read, DynamoDB write/read, and SNS publish permissions
- Logging enabled using **Amazon CloudWatch Logs**

5.3. Data Storage with Amazon DynamoDB

A **DynamoDB table** is created to store inventory data.

- Table name: AppleInventoryTable
- Primary key: Product (string)
- Additional attributes: StoreRegion (string), ProductCount (integer)

Features:

- Automatic scaling to handle varying loads
- Real-time data access for reads and writes
- TTL or secondary indexes can be added for analytics or history tracking (future scope)

5.4. Low-Stock Notification with Amazon SNS

Once the Lambda function identifies products below the threshold, it compiles the product names and quantities into a summary message:

- This message is published to an **Amazon SNS topic**.

- Email subscription is added to the topic for real-time alert delivery.
- SNS ensures that stakeholders are notified promptly to restock items.

5.5. IAM Roles and Security

All interactions between services are secured using **IAM roles** with the principle of **least privilege**:

- Lambda role: Access to read S3, read/write DynamoDB, and publish to SNS.
- S3 bucket policy: Triggers Lambda on object creation.
- SNS topic policy: Allows publish from Lambda and subscription by verified email.

5.6. Monitoring with Amazon CloudWatch

To ensure visibility into the system's behavior and performance:

- CloudWatch logs capture Lambda execution details and errors.
- Metrics such as invocation count, execution duration, and error count are monitored.
- Optional alarms can be configured for failed Lambda executions or missed notifications.

The successful implementation of this serverless system showcases how AWS services can be integrated to create a real-time, low-maintenance, and scalable solution for inventory tracking.

The modular and event-driven approach ensures flexibility for future enhancements such as API endpoints, dashboards, or predictive restocking logic.

6. RESULTS

The implementation of the Serverless Inventory Management System yielded several successful outcomes that align with the project's objectives. The system was tested under multiple scenarios to validate functionality, performance, scalability, and reliability. The following results highlight the effectiveness of using serverless architecture on AWS for real-time inventory monitoring and alerting.

6.1. Automated Inventory Data Processing

When products with quantities below the defined threshold are detected, the Lambda function compiles a summary of low-stock items and sends an email alert via **Amazon SNS**. The emails are delivered to subscribed users almost instantaneously, confirming that the notification mechanism is reliable and timely.

6.2. Real-Time Low-Stock Notifications

When products with quantities below the defined threshold are detected, the Lambda function compiles a summary of low-stock items and sends an email alert via **Amazon SNS**. The emails are delivered to subscribed users almost instantaneously, confirming that the notification mechanism is reliable and timely.

6.3. Scalable and Serverless Architecture

The system handles varying volumes of data without any manual scaling or downtime, thanks to the underlying **serverless AWS infrastructure**. The combination of S3, Lambda, DynamoDB, and SNS ensures that each component functions independently and scales automatically based on demand.

Testing scenarios included:

- Uploading small and large CSV files
- Submitting multiple files in quick succession
- Monitoring invocation logs and latency

All scenarios were handled successfully without performance degradation.

6.4. Secure and Controlled Service Access

Using **AWS IAM roles and policies**, secure access was configured between all services:

- Lambda functions could only access required resources (S3, DynamoDB, SNS)
- S3 bucket events were restricted to specific Lambda triggers
- SNS topic subscriptions were managed through verified email endpoints

This validated that the system follows **best practices for cloud security and permissions**.

6.5. System Monitoring and Logging

Through **Amazon CloudWatch**, the system was monitored for:

- Lambda execution success/failure
- Processing time
- Errors and exception handling

The logs confirmed successful Lambda execution for all test cases and provided helpful debugging information during development and refinement.

6.6. User Experience and Functionality

The system requires minimal user interaction:

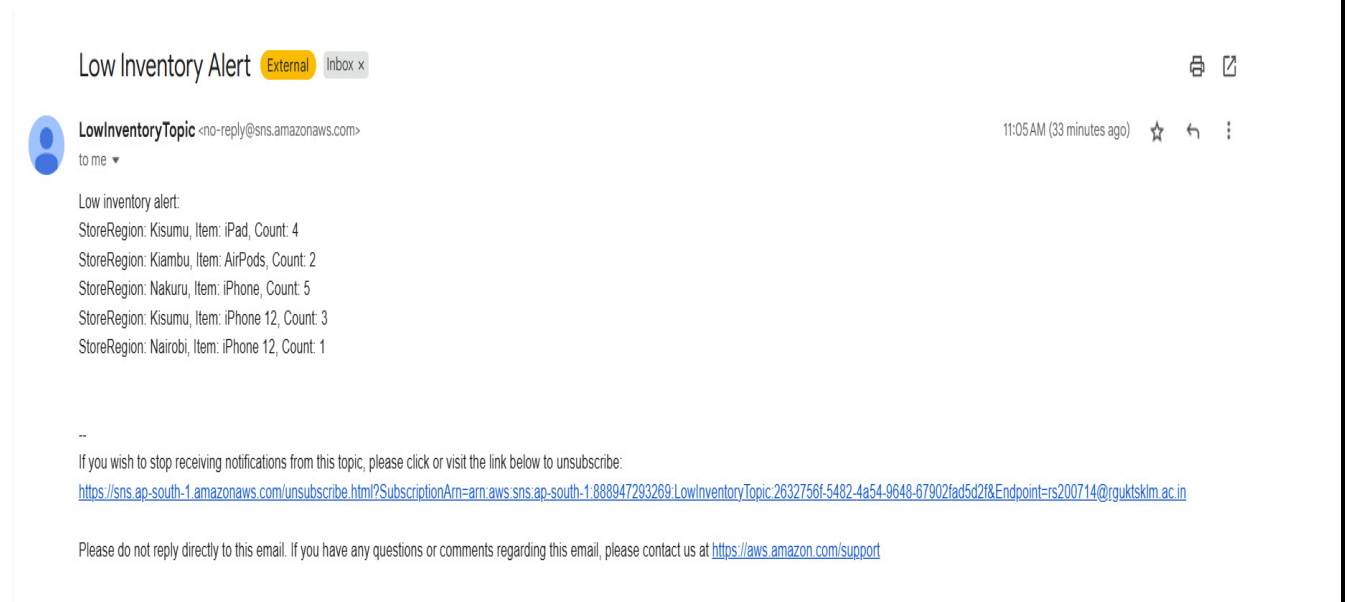
- Uploading a CSV file is the only manual step

- All subsequent actions are fully automated
- Notifications are sent in a clear, human-readable format

This ease of use confirms the system’s potential for **real-world deployment** in retail, warehouse, or inventory management environments.

The overall results demonstrate that the serverless inventory system meets its intended goals of automation, efficiency, scalability, and ease of use. The architecture is stable and ready for real-time operational use, with opportunities for further expansion and integration.

6.7. Screenshots of the output



7.CONCLUSION

The successful development and deployment of the **Serverless Inventory Management System** demonstrate the power and practicality of cloud-native, event-driven architectures in solving real-world business challenges. By leveraging key **AWS services** such as **S3, Lambda, DynamoDB, SNS, and IAM**, the system achieves its core objectives: automating inventory data ingestion, detecting low-stock items, and delivering real-time notifications — all without managing any physical infrastructure.

The project highlights the benefits of **serverless computing**, including **automatic scaling, cost-efficiency, fault tolerance, and reduced operational complexity**. The use of **Lambda functions** allows for on-demand processing of CSV files, while **DynamoDB** provides fast and reliable storage for product data. The **SNS-based notification mechanism** ensures that stakeholders are alerted immediately when inventory levels drop, enabling timely decision-making and restocking.

Throughout the development lifecycle, careful attention was given to **security, performance, and modularity**. The system was designed using **best practices** for IAM permissions, logging via CloudWatch, and monitoring of all integrated services. Testing confirmed the system's ability to process various input sizes and conditions reliably.

Beyond its current functionality, the project lays a strong foundation for future enhancements, such as:

- Adding a **web-based dashboard** for inventory visualization
- Exposing **API endpoints** using Amazon API Gateway
- Integrating **predictive analytics** using Amazon SageMaker for demand forecasting

In conclusion, this project serves as a practical and scalable blueprint for serverless application development, particularly in domains where automation and real-time responsiveness are critical. It showcases how cloud technologies can be strategically combined to build efficient, secure, and future-ready solutions with minimal operational burden.